

MANUALE TECNICO

The Knife



**Università degli studi Insubria
Corso di laurea in informatica(F004)
AA 2025-2026**

**Sviluppato da: Andrea Stagno(756957), Balkaran Singh(758706),
Tommaso Raso(758452), Lorenzo Moneta(756146)**

Indice

1. Introduzione
2. Software design
3. Scelte progettuali
 - 3.1. Modello ER
4. Connessione al database
5. Architettura
6. Strutture dati
7. Algoritmi impiegati
8. Gestione della concorrenza
9. Sicurezza

1.Introduzione

TheKnife è una piattaforma che consente di trovare ristoranti in tutto il mondo, visionare le loro caratteristiche e i servizi che offrono. Per ulteriori informazioni sul funzionamento del programma consultare il manuale utente.

2. Software design

L' applicazione è stata sviluppata utilizzando java 25 e testata su sistemi operativi windows 11 e MacOS.

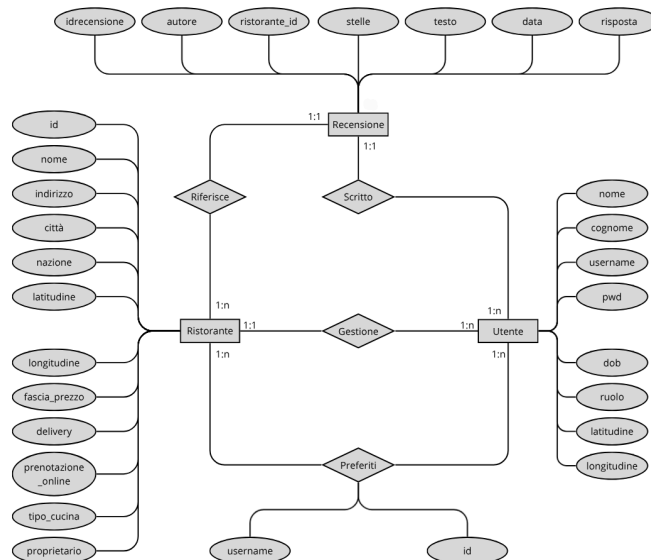
Per la persistenza dei dati è stato usato il DBMS PostgreSQL come richiesto. Per la comunicazione tra client e server è stata utilizzata una connessione a socket e flussi di dati, utilizzando JDBC.

L' interfaccia grafica è stata sviluppata usando vari elementi di Java Swing.

3.Scelte progettuali

3.1 Modello ER

Riportiamo l'immagine del modello ER per completezza, ma se necessario fare riferimento all'immagine presente nella cartella **doc** del progetto.



Nel nostro database abbiamo deciso di inserire 4 entità principali: **Ristorante**, **Utente**, **Recensione** e **Preferiti**.

Partendo dalla prima si può notare che il numero di attributi che possiede è diverso rispetto ai ristoranti nel dataset di partenza, infatti non ci sono

PhoneNumber, Url, Website, Award, GreenStar, Facilities and Services e Description. Abbiamo deciso di fare così per una questione di semplicità e per mettere in evidenza le informazioni che, a nostro avviso, sembravano più importanti. In compenso abbiamo aggiunto diversi attributi : **proprietario**, fondamentale per mostrare i giusti ristoranti agli utenti di tipo Ristoratore; **id**, che permette di identificare univocamente ciascun ristorante e si incrementa automaticamente a ogni inserimento di un nuovo ristorante; **fascia_prezzo**, indica il prezzo medio per quel ristorante; **delivery**, mostra se il ristorante fornisce il servizio di delivery e **prenotazione_online**, analogo a delivery ma per le prenotazioni online.

Per gli attributi aggiunti da noi abbiamo dovuto fare diversi accorgimenti:

- nel dataset originale il campo prezzo era mostrato come una sequenza di simboli, da 1 a 4, che indicavano la valuta del paese a cui appartenevano. Quindi durante l'inserimento nel database abbiamo deciso di tradurre i simboli in un modo più comprensibile: se c'era solo un simbolo (es. \$) generavamo un numero compreso tra 1 e 25, se ce n'erano due un numero compreso tra 26 e 50, se ce n'erano tre tra 51 e 75 e infine se ce n'erano quattro tra 76 e 100. Per generare il numero abbiamo usato la classe Random di Java con il metodo nextInt() inserendo come input il range necessario. Nota: il range è stato scelto in modo arbitrario e non è correlato in nessun modo al ristorante (magari i prezzi potrebbero essere anche sopra ai 100, ma abbiamo scelto questo range per semplicità).
- siccome nel dataset originale non erano presenti le informazioni sulla possibilità del delivery o prenotazioni online, abbiamo utilizzato il metodo nextBoolean().

Continuando con Utente, i campi sono praticamente gli stessi di quelli scritti nelle specifiche fornite, l'unica differenza è che abbiamo deciso di salvare il domicilio di un utente come latitudine e longitudine per semplificare l'esecuzione delle query che richiedono la posizione.

La password prima di venire salvata nel database viene criptata usando l'algoritmo Blowfish e la libreria jbcrypt, e di conseguenza ha lunghezza fissa di 60.

Il campo dob può anche essere null.

Infine il Ruolo può essere solo Cliente o Ristoratore.

Tra le entità aggiunte da noi troviamo per prima Recensione, che ci permette di salvare nel database tutte le recensioni scritte dagli utenti, e Preferiti che non è

una vera e propria entità ma è comunque una tabella a parte in quanto la relazione tra Ristorante e Utente è di tipo n:n.

I vincoli di cardinalità sono:

- Un Ristorante può essere gestito da un solo Utente e un Utente può gestire n Ristoranti. Quindi metteremo in Ristorante la foreign key
- Un Utente può scrivere n Recensioni e una determinata Recensione può essere scritta solo da un Utente. Quindi Recensione conterrà la foreign key
- Un Ristorante può essere riferito in n Recensioni e una determinata Recensione si riferisce a un solo Ristorante. Quindi Recensioni conterrà la foreign key.
- Un Ristorante può essere nei preferiti di n Utenti e un Utente può avere n Ristoranti preferiti. Quindi bisognerà creare una nuova tabella preferiti che conterrà le primary key dei due come foreign key.

Traducendo il modello ER nel modello relazionale avremo:

- Ristorante(id: Int, nome: String, indirizzo: String, città: String, nazione: String, latitudine: Double, longitudine: Double, fascia_prezzo: Int, delivery: Boolean, prenotazione_online: Boolean, tipo_cucina: String, proprietario*: String).
- Utente(nome: String, cognome: String, username: String, pwd: String, dob: Date, ruolo: String, latitudine: Double, longitudine: Double)
- Recensione(idrecensione: Int, autore*: String, ristorante_id*: Int, stelle: Int, testo: String, data: Date, risposta: String)
- Preferiti(username*: String, id*: Int)

I vincoli sulle varie tabelle sono:

- id è primary key per Ristorante e proprietario è una foreign key che punta alla primary key username di Utente.
- username è primary key per Utente e sul ruolo abbiamo un CHECK
- idrecensione è primary key per Recensione, autore è foreign key che punta alla primary key username in Utente e ristorante_id è foreign key che punta alla primary key id in Ristorante. Abbiamo anche un CHECK sul numero di stelle per vedere se è <=5
- la primary key di preferiti è formata da username e id, e entrambi sono foreign key che puntano alle primary key di Utente e Ristorante
- In caso di cancellazione di un ristorante su cui sono state scritte delle recensioni o che era contenuto in una lista dei preferiti, le recensioni verranno eliminate e il ristorante non sarà più nella lista dei preferiti.

I comandi SQL necessari per la creazione del database seguendo tutte le caratteristiche appena descritte saranno:

```
CREATE DATABASE TheKnife
```

```
CREATE TABLE Utente(  
    nome VARCHAR(20) NOT NULL,  
    cognome VARCHAR(20) NOT NULL,  
    username VARCHAR(20) PRIMARY KEY,  
    pwd VARCHAR(60) NOT NULL,  
    dob DATE,  
    ruolo VARCHAR(20) CHECK (ruolo IN ('Cliente', 'Ristoratore') ),  
    latitudine NUMERIC(9,6) NOT NULL,  
    longitudine NUMERIC(9,6) NOT NULL  
)
```

```
CREATE TABLE Ristorante(  
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    nome TEXT NOT NULL,  
    indirizzo TEXT NOT NULL,  
    città TEXT NOT NULL,  
    nazione TEXT NOT NULL,  
    latitudine NUMERIC(9,6) NOT NULL,  
    longitudine NUMERIC(9,6) NOT NULL,  
    fascia_prezzo INT NOT NULL,  
    delivery BOOLEAN NOT NULL,  
    prenotazione_online BOOLEAN NOT NULL,  
    tipo_cucina TEXT NOT NULL,  
    proprietario VARCHAR(50) NOT NULL,  
    CONSTRAINT fk_ristorante_proprietario  
        FOREIGN KEY (proprietario)  
        REFERENCES Utente(username)  
)
```

```
CREATE TABLE Recensione(  
    idrecensione BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    autore VARCHAR(20) NOT NULL,
```

```

    ristorante_id BIGINT,
    stelle INT CHECK (stelle >=1 AND stelle <=5),
    testo TEXT NOT NULL,
    data DATE NOT NULL,
    risposta TEXT DEFAULT NULL,
    CONSTRAINT fk_recensione_autore
        FOREIGN KEY (autore)
        REFERENCES Utente(username),
    CONSTRAINT fk_recensione_ristorante
        FOREIGN KEY (ristorante_id)
        REFERENCES Ristorante(id)
    ON DELETE CASCADE
)

```

```

CREATE TABLE Preferiti(
    username VARCHAR(50),
    ristorante_id BIGINT,
    PRIMARY KEY(username,ristorante_id),
    CONSTRAINT preferiti_ristorante_id_fkey
        FOREIGN KEY (ristorante_id)
        REFERENCES Ristorante(id)
    ON DELETE CASCADE,
    CONSTRAINT preferiti_username_fkey
        FOREIGN KEY (username)
        REFERENCES Utente(username)
)

```

Tutte le query che sono state usate per manipolare i dati sono:

- INSERT INTO utenti (nome, cognome, username, pwd, dob, latitudine,longitudine, ruolo) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
- SELECT pwd,latitudine,longitudine FROM utenti WHERE username = ?
- SELECT * FROM ristorantihekknife ORDER BY id ASC
- SELECT DISTINCT TRIM(x.tipo) AS tipo
FROM ristorantihekknife AS r
CROSS JOIN LATERAL unnest(string_to_array(r.tipo_cucina, ',')) AS x(tipo)
WHERE r.tipo_cucina IS NOT NULL
ORDER BY TRIM(x.tipo)
- INSERT INTO preferiti(username, ristorante_id)" + "VALUES(?, ?)
- DELETE FROM preferiti WHERE ristorante_id = ? AND username = ?

- SELECT r.* FROM preferiti p
JOIN ristorantiithekknife r ON p.ristorante_id = r.id
WHERE p.username = ?
- INSERT INTO recensioni (autore, ristorante_id, stelle, testo, data)
VALUES (?, ?, ?, ?, CURRENT_DATE)
- SELECT idrecensione, autore, ristorante_id, stelle, testo, data, risposta
FROM recensioni
WHERE ristorante_id = ?
ORDER BY data DESC, idrecensione DESC
- UPDATE recensioni SET risposta = ? WHERE idrecensione = ? AND
risposta IS NULL
- DELETE FROM recensioni WHERE idrecensione = ? AND autore = ?
- SELECT COUNT(*) AS totale,
COALESCE(ROUND(AVG(stelle), 2), 0) AS media,
COUNT(CASE WHEN stelle = 5 THEN 1 END) AS s5,
COUNT(CASE WHEN stelle = 4 THEN 1 END) AS s4,
COUNT(CASE WHEN stelle = 3 THEN 1 END) AS s3,
COUNT(CASE WHEN stelle = 2 THEN 1 END) AS s2,
COUNT(CASE WHEN stelle = 1 THEN 1 END) AS s1
FROM recensioni WHERE id_ristorante = ?
- SELECT idrecensione, autore, ristorante_id, stelle, testo, data, risposta
FROM recensioni WHERE autore = ? ORDER BY data DESC, idrecensione
DESC
- UPDATE recensioni SET stelle = ?, testo = ? WHERE idrecensione = ? AND
autore = ?
- INSERT INTO ristorantiithekknife (nome, indirizzo, citta, nazione, latitudine,
longitudine, fascia_prezzo, delivery, prenotazione_online, tipo_cucina,
proprietario) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
- SELECT * FROM utenti WHERE username = ?
- SELECT * FROM ristorantiithekknife WHERE proprietario = ?
- DELETE FROM ristorantiithekknife WHERE id=? AND proprietario=?
- SELECT * FROM ristorantiithekknife WHERE id = ?
- SELECT COUNT(*) as numero_recensioni,
ROUND(AVG(stelle), 2) as media_recensioni
FROM recensioni
WHERE ristorante_id = ?
- Un caso particolare è stato usato durante la ricerca con filtri dei
ristoranti. In quel caso la query è dinamica e si aggiungono condizioni in
base a ciò che ha scelto l'utente

3.2 UML

Facciamo riferimento all'immagine dello schema UML nella cartella **doc**.

In clientTK abbiamo deciso di mettere solo due attributi: SERVER_IP e SERVER_PORT che sono final così che non possano essere modificati; e un metodo inviaRichiesta() che si limita solo a mandare le proprie richiesta sullo stream collegato al Thread perchè il client non deve conoscere nulla sul funzionamento del backend dell'applicazione.

In serverTK abbiamo tre attributi che salvano le informazioni sul database a cui bisogna connettersi e un metodo accessoDB che richiede queste informazioni. Nel main il server crea la socket e si mette in attesa, poi quando riceve una richiesta di connessione la accetta e avvia il thread.

In serverSlave, cioè il Thread, ci occupiamo solo di ricevere le richieste del client e decodificarle in base a un identificativo presente nel messaggio, per poi chiamare il metodo giusto per svolgere l'operazione. Tutti i metodi hanno la stessa struttura: fanno una chiamata alla classe GestoreDatabase e poi spediscono il risultato ricevuto al client.

In GestoreDatabase abbiamo inserito tutti i metodi che manipolano i dati del db. Tutti i metodi hanno più o meno la stessa struttura: ricevono in input i dati del db a cui collegarsi e qualche altra informazione in più che dipende da cosa bisogna fare, poi si connettono al db; e ritornano il risultato dell'operazione, che potrebbe essere un insieme di dati dal db se si stava facendo una select oppure una stringa che indica il risultato dell'operazione se si stava facendo una insert. In ciascun caso facciamo utilizzo dei preparedStatement.

Nelle varie finestre della GUI abbiamo semplicemente dei metodi che

- aprono altre finestre, eventualmente passando delle informazioni
- che prendono i dati inseriti in input dall'utente e dopo averli controllati fanno una richiesta al thread
- dopo aver ricevuto il risultato della richiesta aggiornano i dati che l'utente sta visualizzando.
- sistemano alcuni elementi visibili all'utente

Nelle classi Utente, Ristorante e Recensione abbiamo semplicemente gli attributi, il/i costruttori e i metodi get e set.

4. Connessione al database

La connessione al database è gestita dall'API JDBC tramite *DriverManager* e il driver JDBC specifico di PostgreSQL.

La connessione con il database è aperta solo dalla classe *GestoreDatabase* nel momento in cui è richiesta una operazione riguardante il database.

5. Architettura

L'applicazione utilizza l'architettura client-server, dove abbiamo 3 ruoli principali:

- Il Server, che si occupa di connettersi al database per la prima volta e di rimanere in attesa delle connessioni da parte dei client e quando ne arriva una, avvia un Thread per soddisfare le richieste dei Client.
- Il Client interagisce con il sistema tramite la GUI, navigando tra varie finestre distinte per facilitare l'utilizzo dell'applicazione.
- Il Thread è il cuore del sistema in quanto permette a ciascun client di reperire le informazioni presenti sul database. Ogni volta che il client esegue un'operazione (si faccia riferimento al manuale utente per vedere cosa si può fare), manda una richiesta specifica al Thread che la decodifica e svolge ciò che viene richiesto. Per svolgere il suo lavoro il Thread deve interfacciarsi con il db utilizzando la classe *GestoreDatabase* che contiene tutti i metodi necessari per manipolare i dati presenti sul database. Dopo aver ottenuto i dati dal *GestoreDatabase*, il Thread li rispedisce al Client, dove la GUI si occuperà di mostrarli al client e consultarli.

Per gestire la comunicazione tra Client e Thread si fa uso delle socket e dei relativi stream di I/O. I tipi di messaggi che vengono inviati sono principalmente Object che contengono l'identificatore dell'azione da eseguire ed eventualmente dei dati necessari per svolgere l'azione; ma non solo, si possono inviare anche istanze di classi come Utente o Ristorante, che di conseguenza dovranno implementare l'interfaccia *Serializable*.

6. Strutture dati

Nell'applicazione non è stato necessario utilizzare strutture dati particolarmente complicate, abbiamo fatto uso principalmente di liste (LinkedList e ArrayList).

Nella GUI sono presenti principalmente JFrame, JPanel, JTable, JDialog per impostare in modo chiaro le informazioni visibili a schermo.

7. Algoritmi impiegati

Dato che la logica dell'applicazione è relativamente semplice non sono stati usati algoritmi particolarmente complessi, per la manipolazione dei dati per motivi di semplicità ed efficienza abbiamo lasciato gestire il tutto al DBMS, di particolare importanza sono la funzione *ricercaRistoranti* che consente di generare una query SQL applicando dei filtri personalizzati e la funzione *ricercaPosizione* che consente di ottenere i valori di latitudine e longitudine corrispondenti ad una località (città e nazione) inserita. Per calcolare la distanza tra due punti abbiamo utilizzato la formula di haversine direttamente nella clausola WHERE della query:

```
String formulaDistanza = "( 6371 * acos( least(1.0, cos( radians(?) ) * cos( radians( r.latitudine ) ) * cos( radians( r.longitudine ) - radians(?) ) + sin( radians(?) ) * sin( radians( r.latitudine ) ) ) ) ) )";
```

```
query.append(" AND ").append(formulaDistanza);
```

Gestione della posizione

Per gestire la posizione degli Utenti o per filtrare i ristoranti in base alla distanza tra loro e gli Utenti, abbiamo utilizzato il servizio **Nominatim** di OpenStreetMap per ottenere le coordinate geografiche in base alla via e/o città e alla nazione.

Durante l'accesso come Ospite, registrazione o nella creazione di un ristorante l'utente deve inserire delle informazioni relative alla posizione come via e/o città e nazione per poter ottenere le coordinate geografiche.

Il servizio prende in input le informazioni specificate qui sopra e le inserisce nel link di base <https://nominatim.openstreetmap.org/search> nel formato giusto, specificando di restituire solo il risultato più rilevante e il formato, che sarà JSON. In seguito si crea una HTTP request di tipo GET utilizzando il link appena creato, specificando un User-Agent(policy del servizio). Si crea un HttpClient

per inviare la richiesta e si salva la risposta come stringa. Si utilizza la libreria Jackson per creare un ObjectMapper e usare il metodo `readTree` per trasformare la risposta in un albero di nodi. Infine se la risposta è vuota, si ritorna null, altrimenti si estraggono le coordinate e si ritornano quelle.

8. Gestione concorrenza

La gestione della concorrenza nell'accesso al database è affidata principalmente al DBMS PostgreSQL, che garantisce l'esecuzione concorrente delle transazioni attraverso i propri meccanismi interni. Per architettura ogni thread andrà a creare la propria connessione con il DBMS, in questo modo il DBMS è in grado di gestire la concorrenza tra le varie connessioni simultanee, in fine dato che i metodi di accesso al database sono metodi statici della classe *GestoreDatabase* abbiamo reso i metodi che modificano le tabelle *Synchronized*.

9. Sicurezza

La password richiesta dal sistema al momento della registrazione deve rispettare le seguenti caratteristiche:

- Almeno otto caratteri
- Almeno un numero
- Almeno una lettera maiuscola
- Almeno un carattere tra `[!@#$%^&*()_+ - = [ ] { } ; ' : " \ | , . < > / ?]`

Al momento del salvataggio nel database la password viene criptata con la seguente operazione:

```
BCrypt.hashpw(passwordInChiaro, BCrypt.gensalt());
```

Il risultato di questa operazione viene salvato nel database, in fase di login l'hash viene recuperato dal database e la validità della password viene verificata nel seguente modo:

```
BCrypt.checkpw(passwordInChiaro, hashSalvato
```

SQL Injection

Per evitare di subire SQL Injection quando facciamo delle query sul database facciamo uso dei PreparedStatement, un esempio di query potrebbe essere `DELETE FROM preferiti WHERE ristorante_id = ? AND username = ?` dove poi bisognerà fare

```
PreparedStatement pstmt = conn.prepareStatement(query))  
pstmt.setString(2,user);  
pstmt.setInt(1,ristorante)
```